ИТМО

Факультет программной инженерии и компьютерной техники

Встроенные системы

Мастер-класс №3

«Программирование микроконтроллеров с операционной системой (FreeRTOS)»

Вариант 2 — «Тетрис»

Выполнили:
Лукина Татьяна Александровна
Антонова Анна Игоревна
466546, 465026, P3332

Преподаватель:
Быковский Сергей Вячеславович

Санкт-Петербург

2026

Оглавление

Цель и задачи работы *3*

Диаграмма последовательностей *4*

Описание процессов *6*

Руководство пользователя *9*

Код *14*

Цель и задачи работы

Цель: разработать для микроконтроллера STM32F427VITx игру «Тетрис», работающую под управлением FreeRTOS, с вводом с матричной клавиатуры через расширитель PCA9538 и выводом игрового состояния на OLED-дисплей 128 × 64 точки.

Задачи работы:

- инициализировать микроконтроллер, GPIO и интерфейс I2C1, затем запустить планировщик FreeRTOS;
- создать три задачи: InputTask для опроса клавиатуры, GameTask для изменения игрового состояния и DisplayTask для вывода на OLED;
- передавать дискретные события клавиш через очередь key_queue и исключить повторную отправку события при удержании клавиши;
- защитить общее игровое состояние мьютексом game_mutex, а общую шину I2C1 — мьютексом i2c_mutex;
- реализовать семь тетромино, повороты, перемещение, ускоренное падение, проверку столкновений и фиксацию фигур;
- реализовать удаление заполненных строк, подсчёт очков и линий, рекорд текущего сеанса и условие окончания игры;
- реализовать экраны меню, игры, паузы и результата, а также управление с клавиатуры 4 × 3;
- подготовить диаграмму последовательностей, описание взаимодействия задач и руководство пользователя.

Диаграмма последовательностей

На рисунках 1 и 2 показана логическая последовательность вызовов. Все три задачи исполняются планировщиком поочерёдно на одном ядре Cortex-M4; вертикальные линии задач не обозначают отдельные физические ядра.

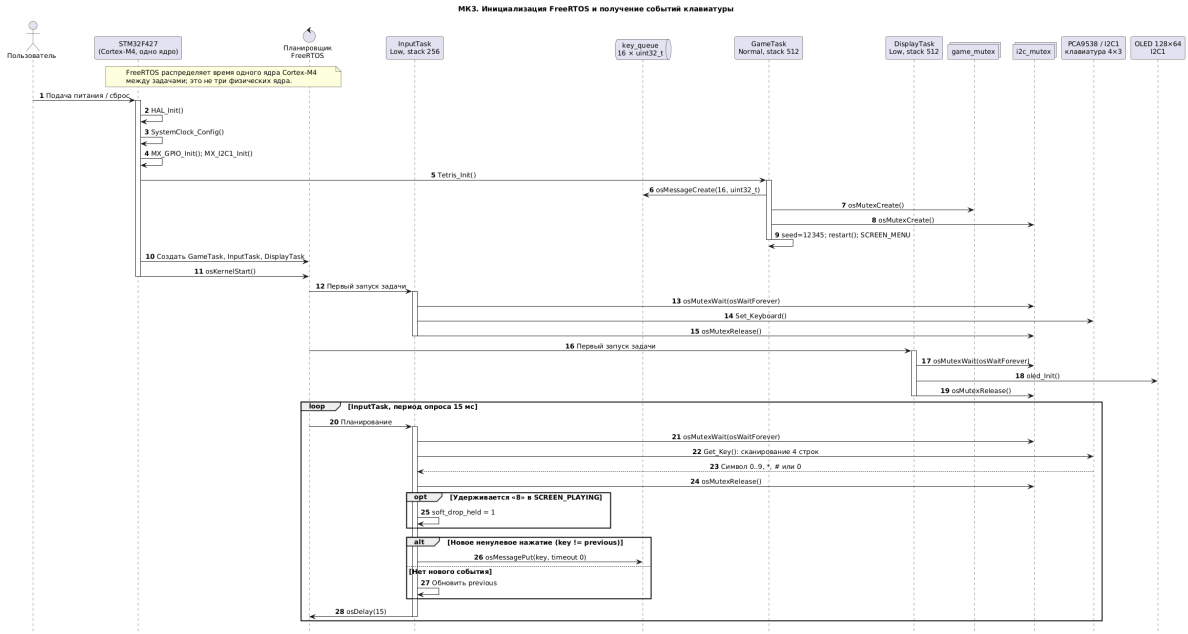


Рисунок 1 — Инициализация FreeRTOS и получение событий клавиатуры

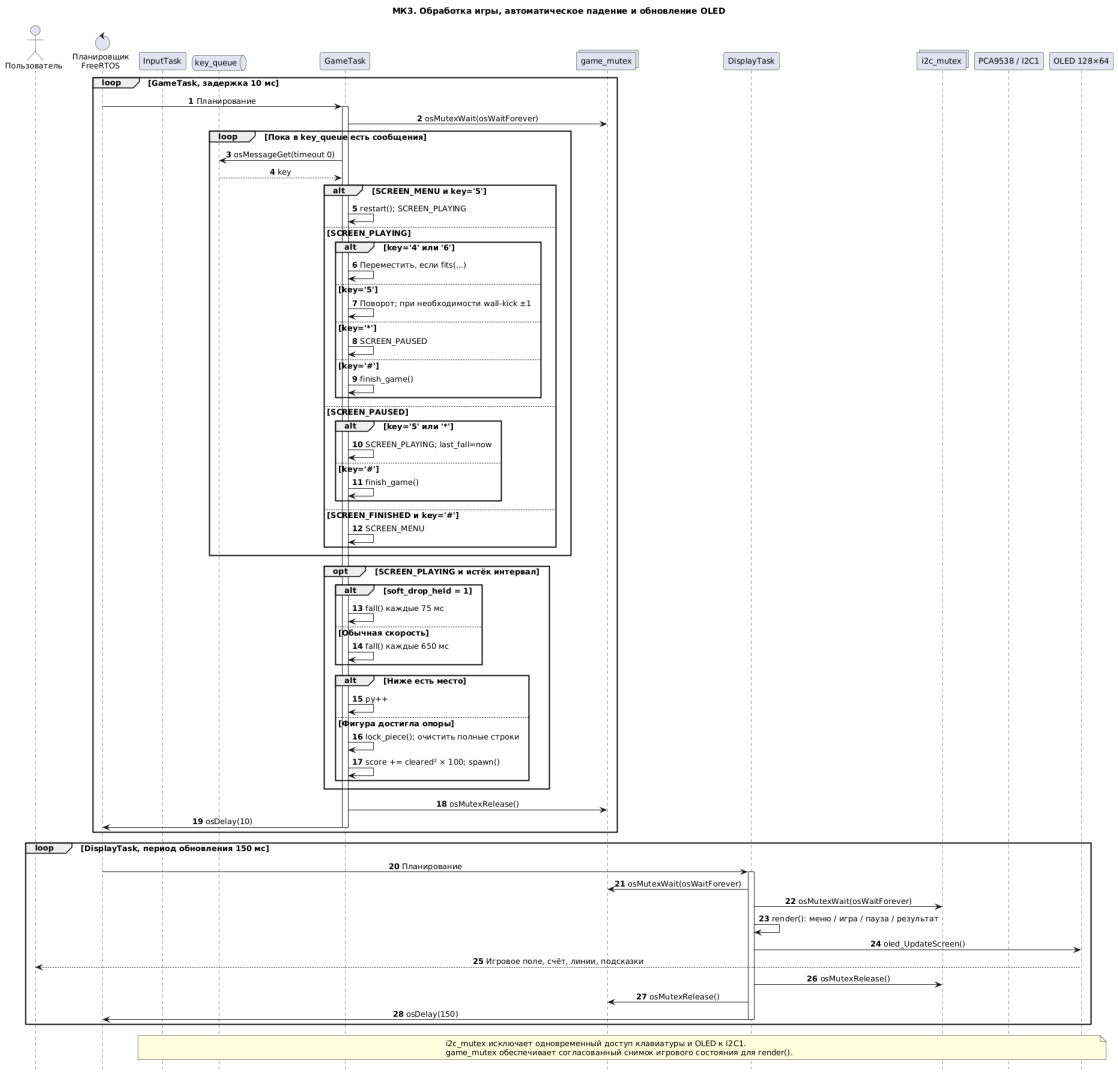


Рисунок 2 — Обработка игры и обновление OLED

Описание процессов

Инициализация микроконтроллера, ядра FreeRTOS и ресурсов

После сброса main() вызывает HAL_Init(), SystemClock_Config(), MX_GPIO_Init() и MX_I2C1_Init(). Интерфейс I2C1 настроен на частоту 100 кГц; в проекте линии SCL и SDA назначены выводам PB8 и PB9. Затем Tetris_Init() создаёт очередь key_queue на 16 сообщений типа uint32_t и два мьютекса: game_mutex и i2c_mutex. Начальное значение генератора фигур равно 12345, игровое поле очищается функцией restart(), создаётся первая фигура, а отображаемым состоянием устанавливается SCREEN_MENU.

До osKernelStart() создаются GameTask с приоритетом osPriorityNormal и стеком 512, InputTask с приоритетом osPriorityLow и стеком 256, а также DisplayTask с приоритетом osPriorityLow и стеком 512. STM32F427 содержит одно ядро Cortex-M4: FreeRTOS не запускает задачи на разных ядрах, а переключает их в соответствии с приоритетами, готовностью и вызовами osDelay().

InputTask: опрос клавиатуры и формирование событий

В начале InputTask функция Set_Keyboard() вызывается под защитой i2c_mutex. Далее задача циклически получает тот же мьютекс, вызывает Get_Key() и освобождает мьютекс. Драйвер kb.c сканирует четыре строки матрицы 4 × 3 через PCA9538 по I2C1 и преобразует активный столбец в символ из раскладки «1 2 3 / 4 5 6 / 7 8 9 / * 0 #». После каждой итерации выполняется osDelay(15).

Событие помещается в key_queue только для нового ненулевого значения key, отличного от previous. Поэтому удерживаемая кнопка не создаёт непрерывный поток одинаковых сообщений. Отдельного временного алгоритма подавления дребезга в исходном коде нет; используются период опроса 15 мс и фиксация фронта по previous. Признак soft_drop_held обновляется независимо: он равен единице, пока удерживается «8» в состоянии SCREEN_PLAYING, что позволяет ускорять падение без многократной постановки клавиши в очередь.

Очередь key_queue и обработка клавиш в GameTask

GameTask каждые 10 мс захватывает game_mutex и без ожидания извлекает из key_queue все накопленные сообщения вызовами osMessageGet(..., 0). Очередь отделяет сравнительно частый опрос клавиатуры от игровой логики: InputTask является производителем копий значений uint32_t, GameTask — единственным потребителем. Если очередь заполнена, osMessagePut(..., 0) не блокирует InputTask.

В меню клавиша «5» вызывает restart() и переводит систему в SCREEN_PLAYING. Во время игры «4» и «6» перемещают фигуру влево и вправо, если fits() подтверждает допустимость позиции; «5» поворачивает фигуру, а при столкновении выполняется попытка сдвига на одну клетку

влево или вправо. «*» включает SCREEN_PAUSED, «#» завершает игру. На паузе «5» или «*» возобновляют игру и обновляют last_fall, а «#» завершает её. На экране результата «#» возвращает меню.

Автоматическое и ускоренное падение

После обработки очереди GameTask проверяет время osKernelSysTick(). Нормальный интервал FALL_MS равен 650 мс, ускоренный FAST_MS — 75 мс. При изменении режима ускорения last_fall сбрасывается, чтобы исключить скачок по старому интервалу. Когда интервал истёк, fall() пытается увеличить py. Если нижняя позиция недопустима, вызывается lock_piece(). После освобождения game_mutex задача выполняет osDelay(10).

Модель фигур, проверка столкновений и фиксация

Игровое поле board имеет размер 10 × 16 клеток, каждая клетка занимает 4 × 4 пикселя OLED. Семь тетромино I, O, T, S, Z, J и L заданы 16-битными масками 4 × 4. shape_at() вычисляет элемент маски с учётом одного из четырёх поворотов. fits() запрещает выход за левую, правую и нижнюю границы и пересечение с уже занятыми клетками; отрицательная координата y для новой фигуры разрешена.

spawn() обновляет линейный конгруэнтный генератор, выбирает тип фигуры, устанавливает rotation = 0, px = 3 и py = −1. Если новая фигура сразу не помещается, вызывается finish_game(). Перед записью фигуры lock_piece() проверяет, что вся она вошла в видимую область. Зафиксированной фигуре назначается свободный идентификатор 1…254; значение 255 зарезервировано для активной фигуры при отрисовке. Уникальные идентификаторы позволяют визуально соединять только клетки одной фигуры.

Удаление строк, очки и завершение игры

После фиксации строки проверяются снизу вверх. Полная строка удаляется, все строки выше сдвигаются вниз, а верхняя строка очищается; повторная проверка той же позиции позволяет удалить несколько соседних заполненных строк. Счётчик lines увеличивается на число удалённых строк, а score — на cleared² × 100: 100, 400, 900 или 1600 очков за одновременное удаление одной, двух, трёх или четырёх строк. Затем создаётся следующая фигура.

finish_game() обновляет record_score, если текущий результат больше, выключает ускоренное падение и устанавливает SCREEN_FINISHED. Рекорд хранится только в оперативной памяти и после сброса питания не сохраняется. Игра также завершается, если для новой или фиксируемой фигуры отсутствует допустимая позиция либо свободный идентификатор.

DisplayTask, буфер OLED и синхронизация I2C1

DisplayTask один раз вызывает oled_Init() под i2c_mutex, затем каждые 150 мс захватывает game_mutex, после него i2c_mutex, вызывает render() и

передаёт буфер на OLED функцией oled_UpdateScreen(). Мьютексы освобождаются в обратном порядке. game_mutex обеспечивает согласованный снимок поля, текущей фигуры, счёта и состояния экрана; i2c_mutex исключает одновременные транзакции OLED и PCA9538 по общей шине I2C1. Во всех местах сохраняется единый порядок захвата, поэтому циклического ожидания между этими задачами не создаётся.

В игровом режиме render() копирует board во временный массив picture и накладывает активную фигуру с идентификатором 255. Пустые клетки показаны точками, занятые — блоками 3 × 3 пикселя; общие границы клеток одной фигуры дорисовываются, формируя цельный силуэт. Поле шириной 40 пикселей начинается с координаты x = 2, рядом выводятся TETRIS, счёт S, число линий L и подсказки «4/6 MOVE», «5ROT 8FAST», «*PAUSE #END». Отдельные экраны показывают меню, паузу и результат с рекордом.

Описание отражает структуру и алгоритмы активного файла Application/User/Core/tetris.c, включённого в objects.list сборки. Экспериментальные файлы каталога Application/variants в отчёт не включены. Аппаратные испытания в предоставленных материалах не зафиксированы, поэтому фактическую работу клавиатуры и OLED следует подтвердить на стенде.

Руководство пользователя

1. Включение и появление меню

Подайте питание на стенд. После инициализации на OLED должно появиться меню «TETRIS», строка «5 - START» и значение «BEST». До запуска игры поле не отображается.

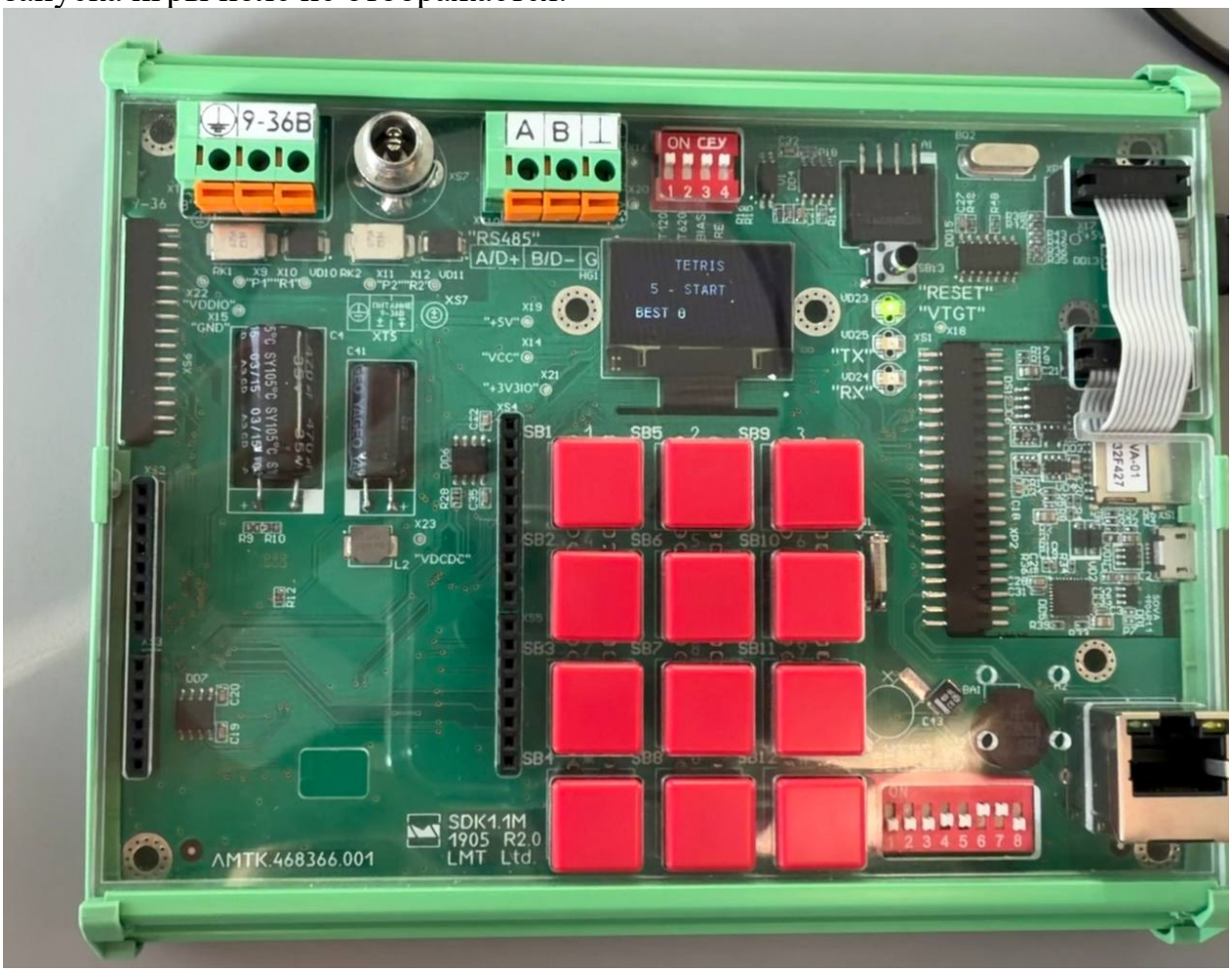


Рисунок 3 — Меню игры после включения

2. Запуск новой игры

Нажмите «5». Поле очищается, счёт и число линий устанавливаются в ноль, создаётся первая фигура и открывается игровой экран. Справа должны быть видны счёт, линии и подсказки управления.

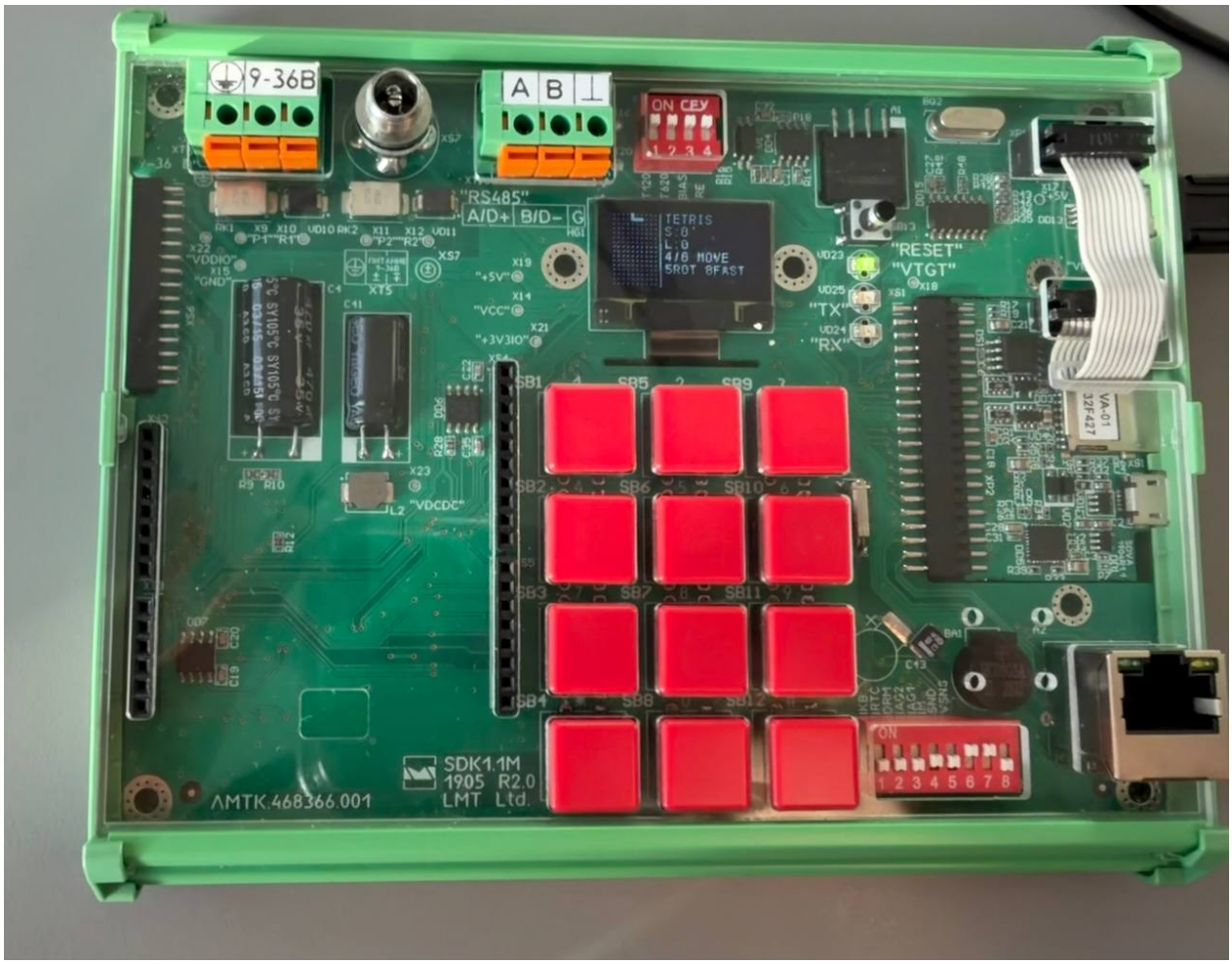


Рисунок 4 — Начало новой игры

3. Перемещение фигуры влево

Кратко нажмите «4». Активная фигура сдвинется на одну клетку влево, если новая позиция находится в границах поля и не пересекает зафиксированные блоки.

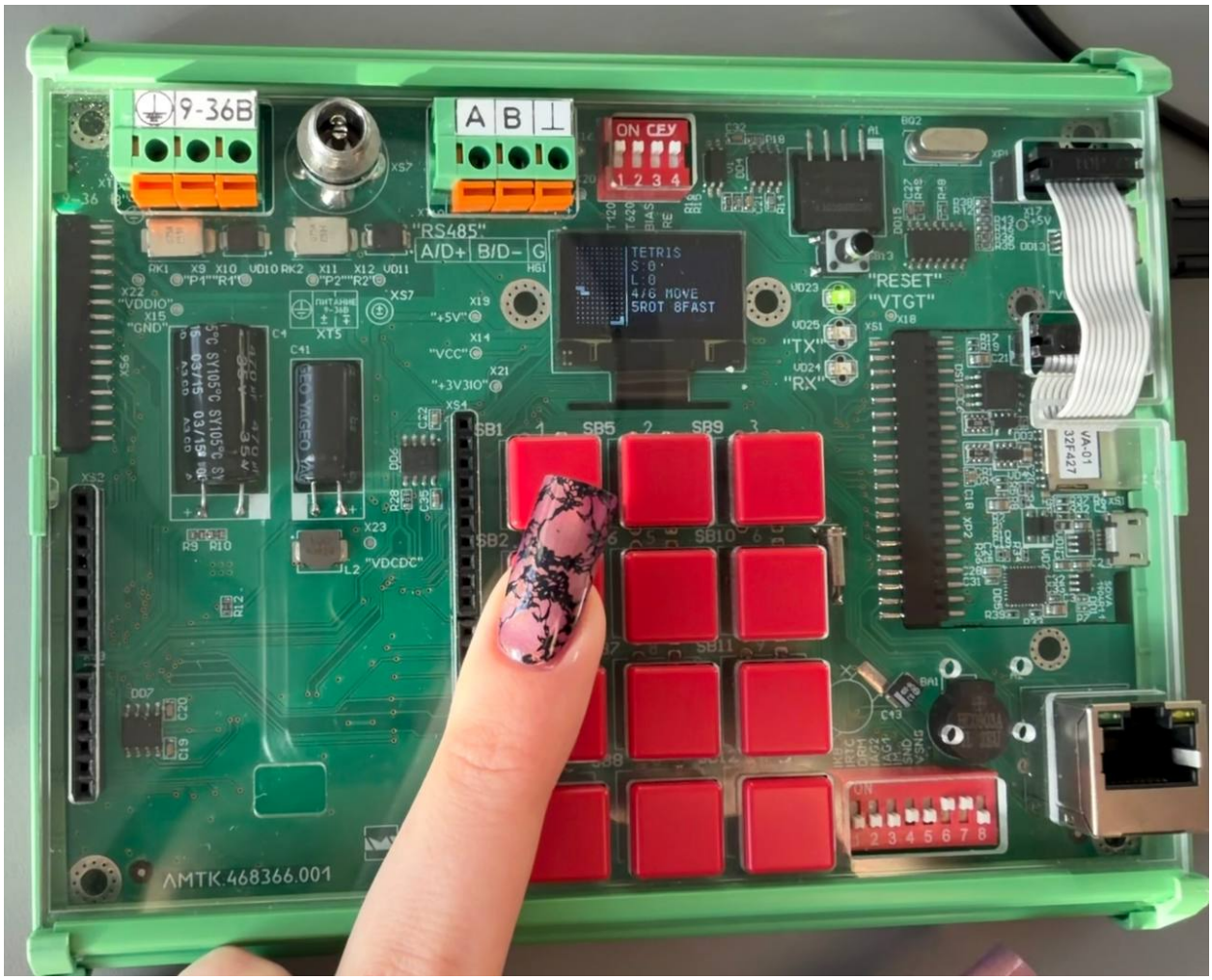


Рисунок 5 — Перемещение фигуры влево

4. Перемещение фигуры вправо

Кратко нажмите «6». Активная фигура сдвинется на одну клетку вправо при допустимой позиции. Удержание не вызывает серии шагов: новое событие создаётся после отпускания и повторного нажатия.

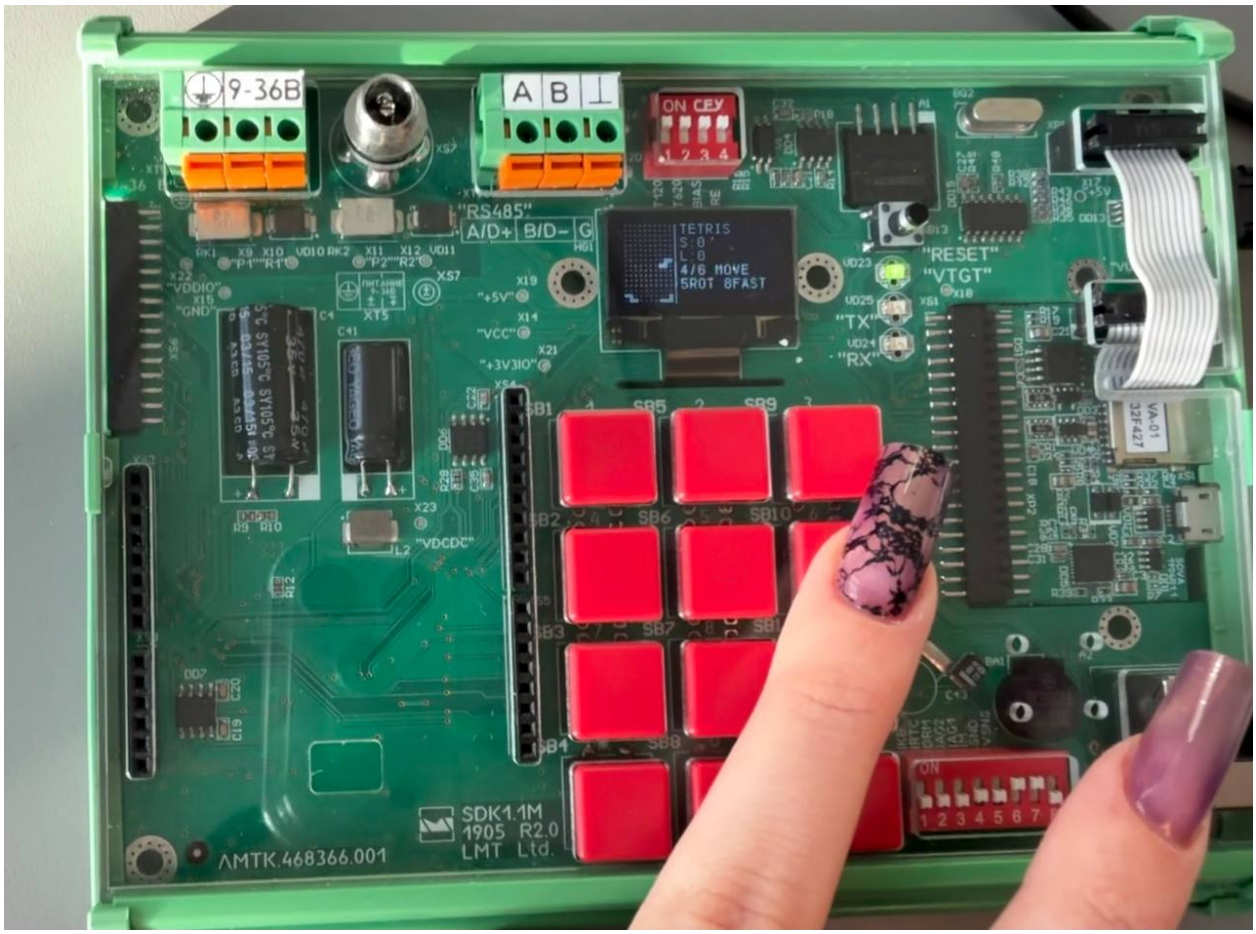


Рисунок 6 — Перемещение фигуры вправо

5. Поворот фигуры

Нажмите «5» во время игры. Фигура поворачивается на 90°. Если поворот на месте невозможен, программа дополнительно пробует сдвиг на одну клетку влево, затем вправо; при отсутствии допустимого варианта положение не меняется.

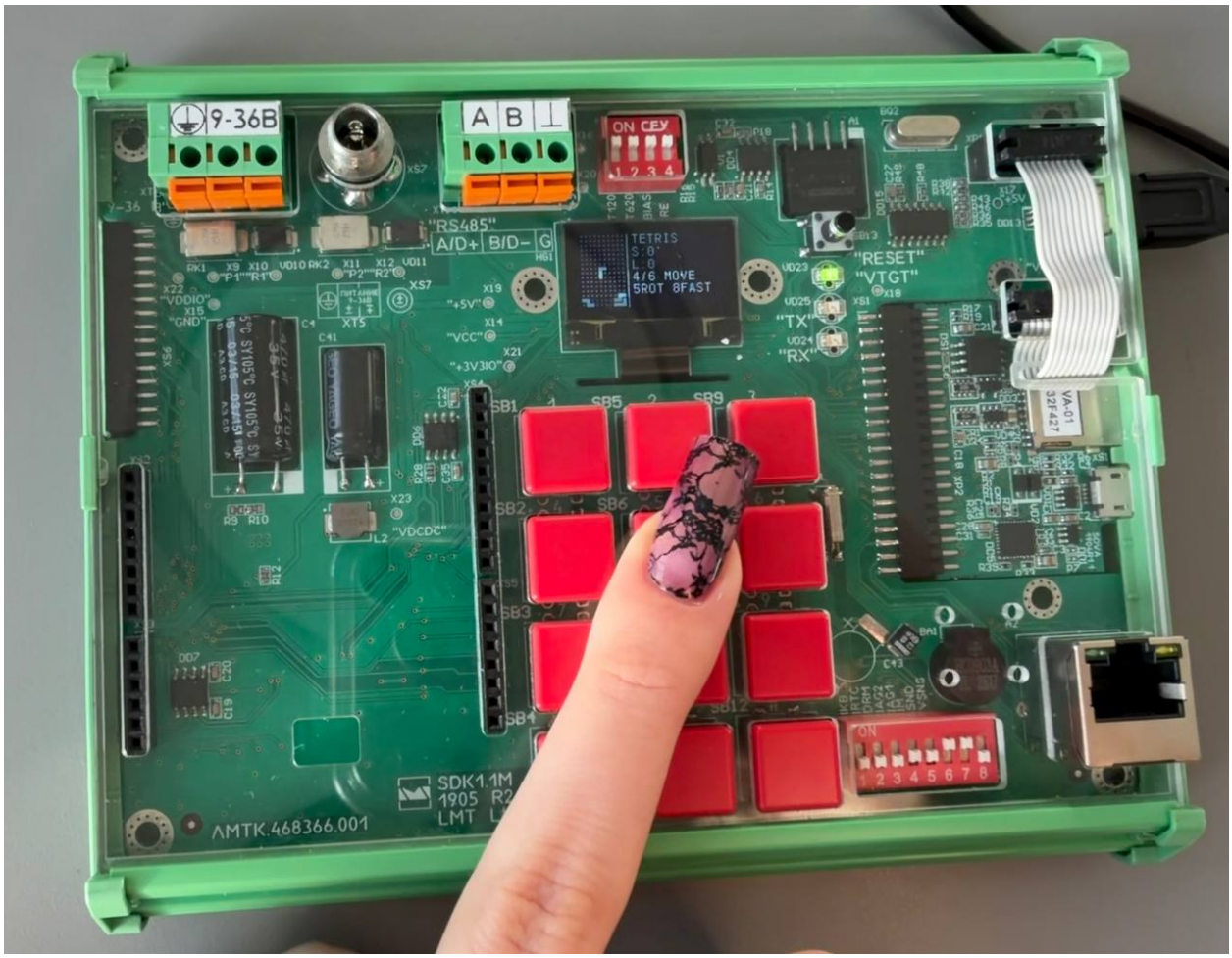


Рисунок 7 — Поворот активной фигуры

6. Ускоренное падение

Нажмите и удерживайте «8». Пока клавиша удерживается, шаг падения выполняется примерно каждые 75 мс вместо 650 мс. Отпустите «8», чтобы вернуться к обычной скорости.

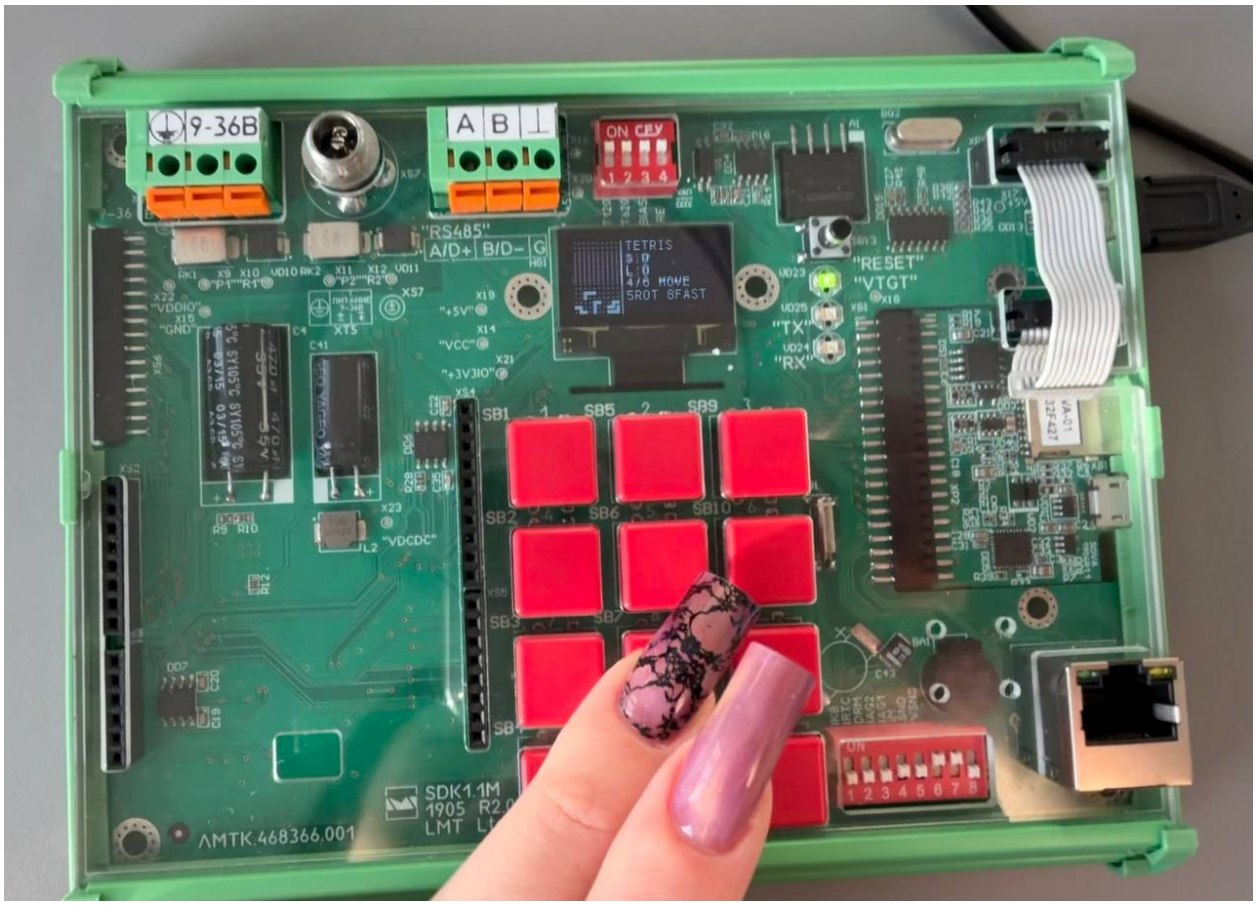


Рисунок 8 — Ускоренное падение при удержании «8»

7. Фиксация фигуры и удаление строк

Доведите фигуру до нижней границы или до другого блока. Она фиксируется автоматически, после чего создаётся следующая. При полном заполнении горизонтальной строки она удаляется; показатели S и L обновляются.

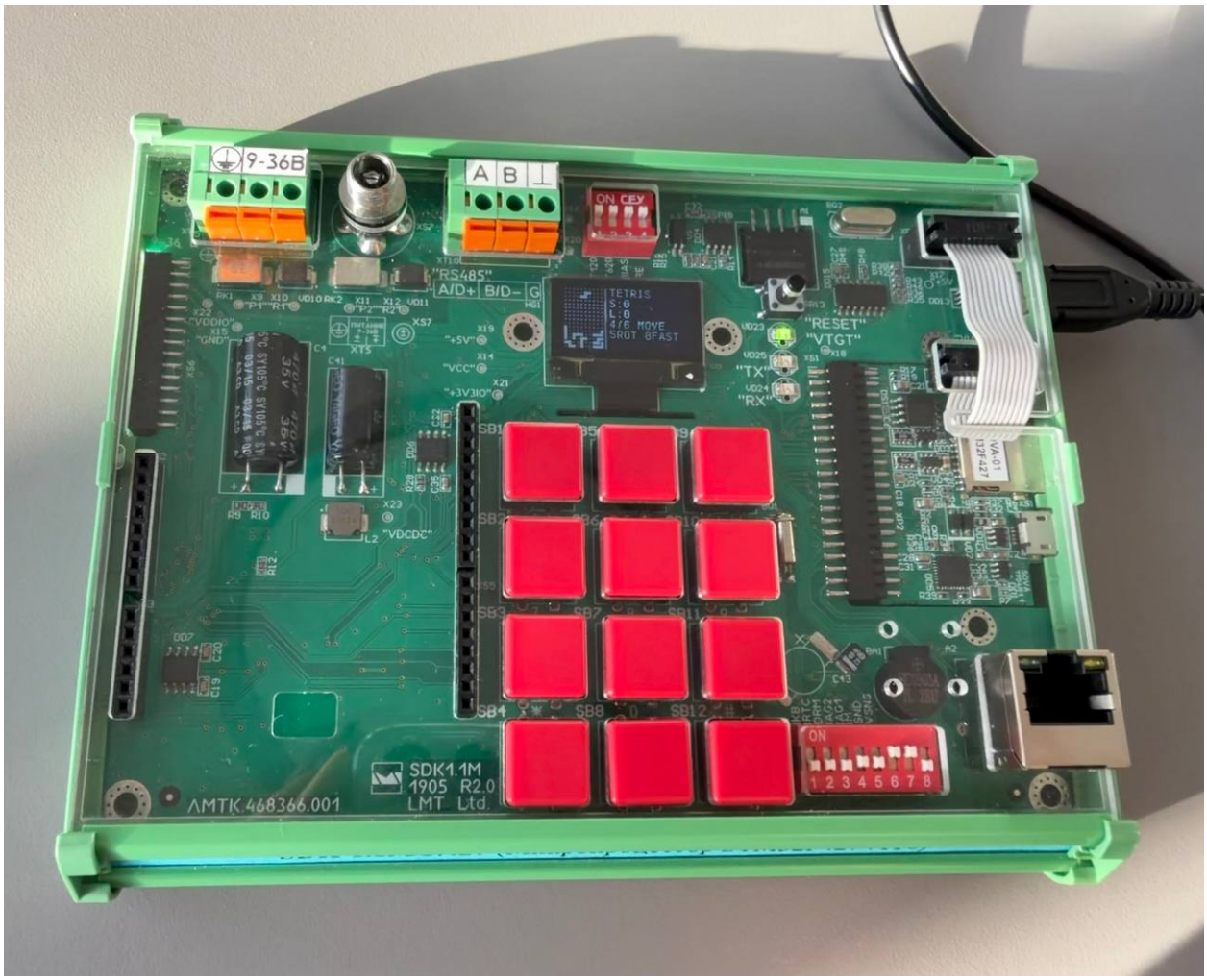


Рисунок 9 — Фиксация фигуры и изменение счёта

8. Пауза и продолжение

Нажмите «*» во время игры. OLED покажет «PAUSE», счёт, линии и подсказки «5 - RESUME» и «# - END». Для продолжения нажмите «5» или «*».

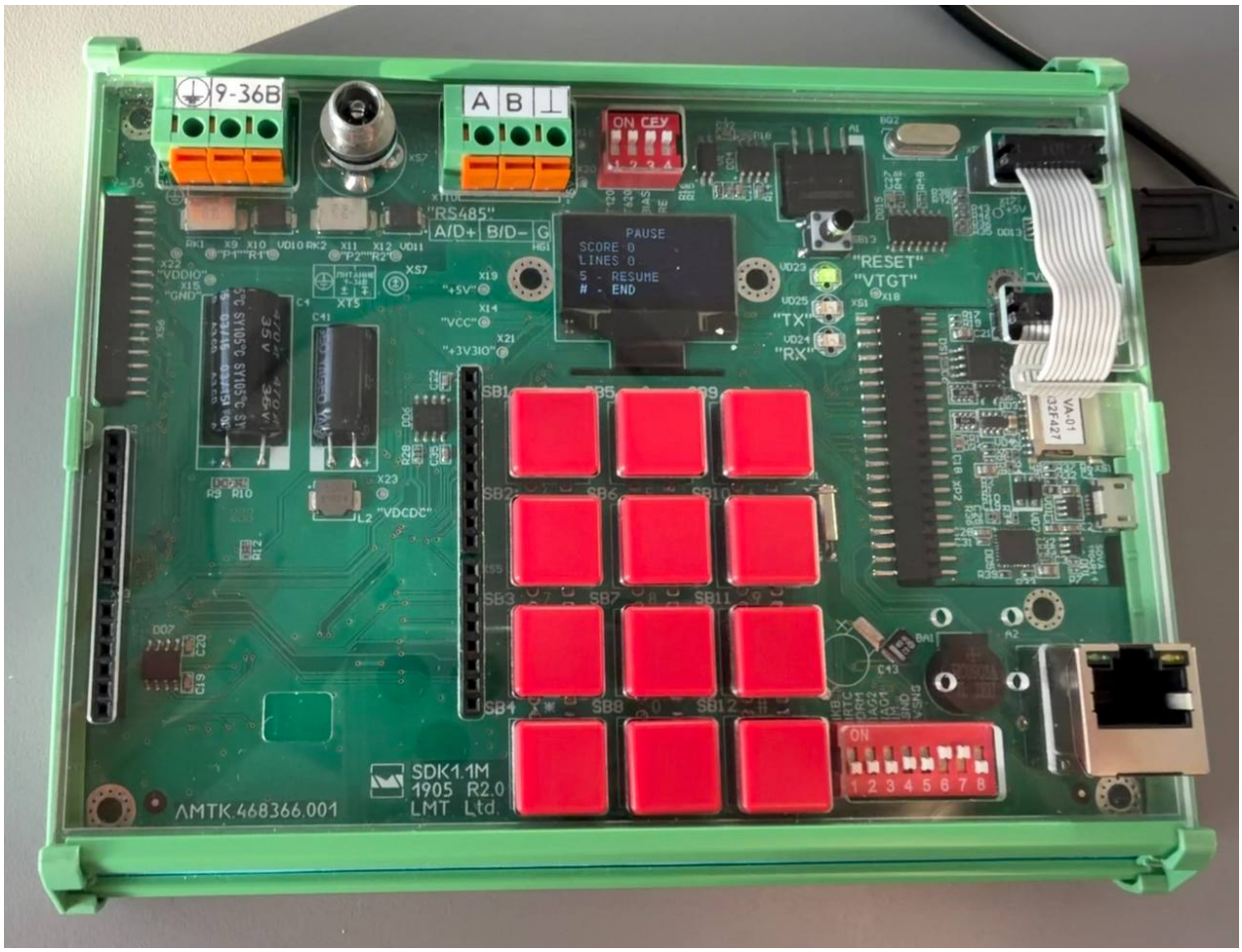


Рисунок 10 — Экран паузы и команды продолжения

9. Досрочное завершение игры

Во время игры или на паузе нажмите «#». Появится экран «GAME OVER» с текущими SCORE, LINES и BEST. Такой же экран появляется автоматически, когда новая фигура не может быть размещена.

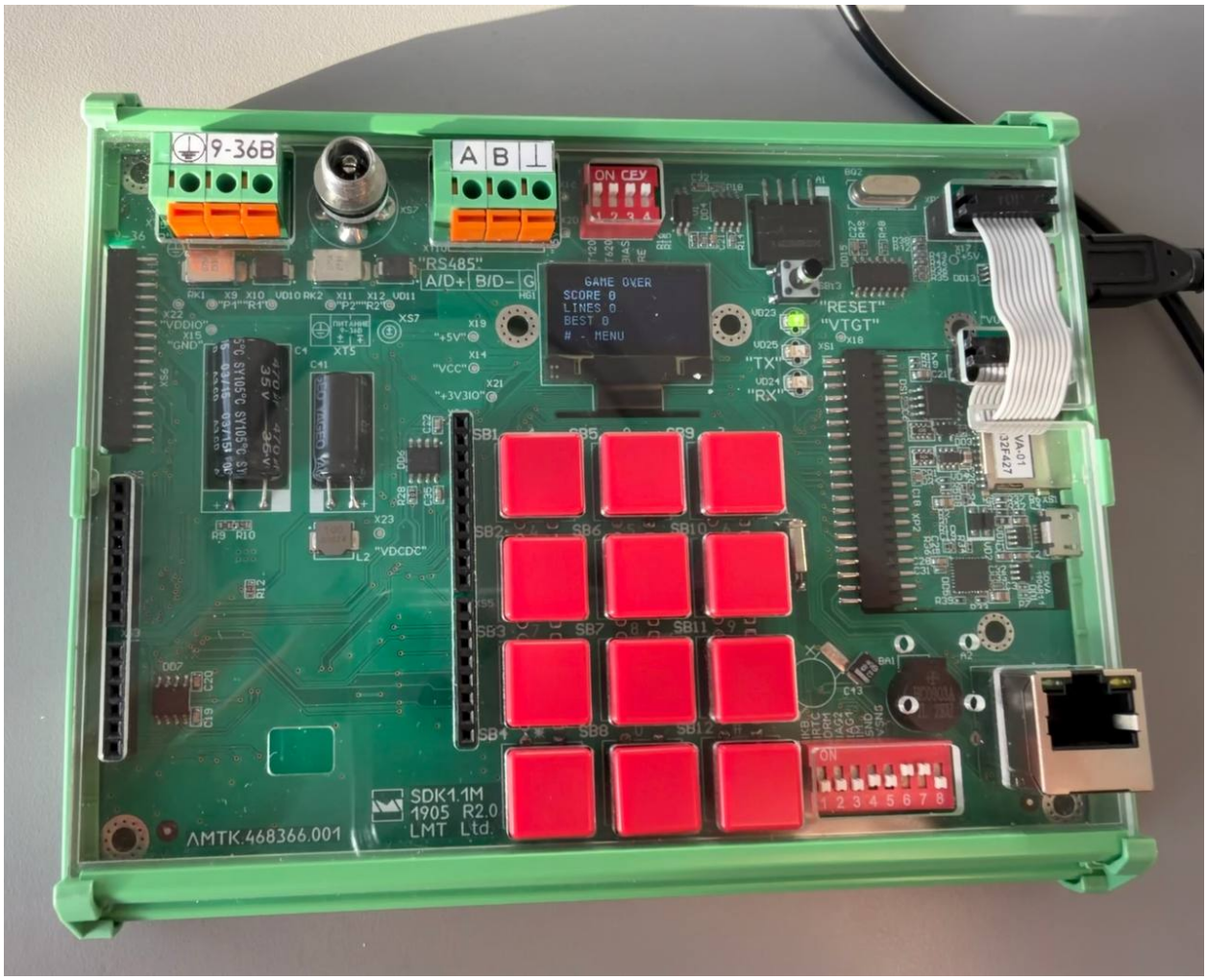


Рисунок 11 — Завершение игры и итоговый результат

10. Возврат в меню

На экране результата нажмите «#». Система вернётся в меню. Значение BEST сохраняется до сброса питания; для следующей игры снова нажмите «5».

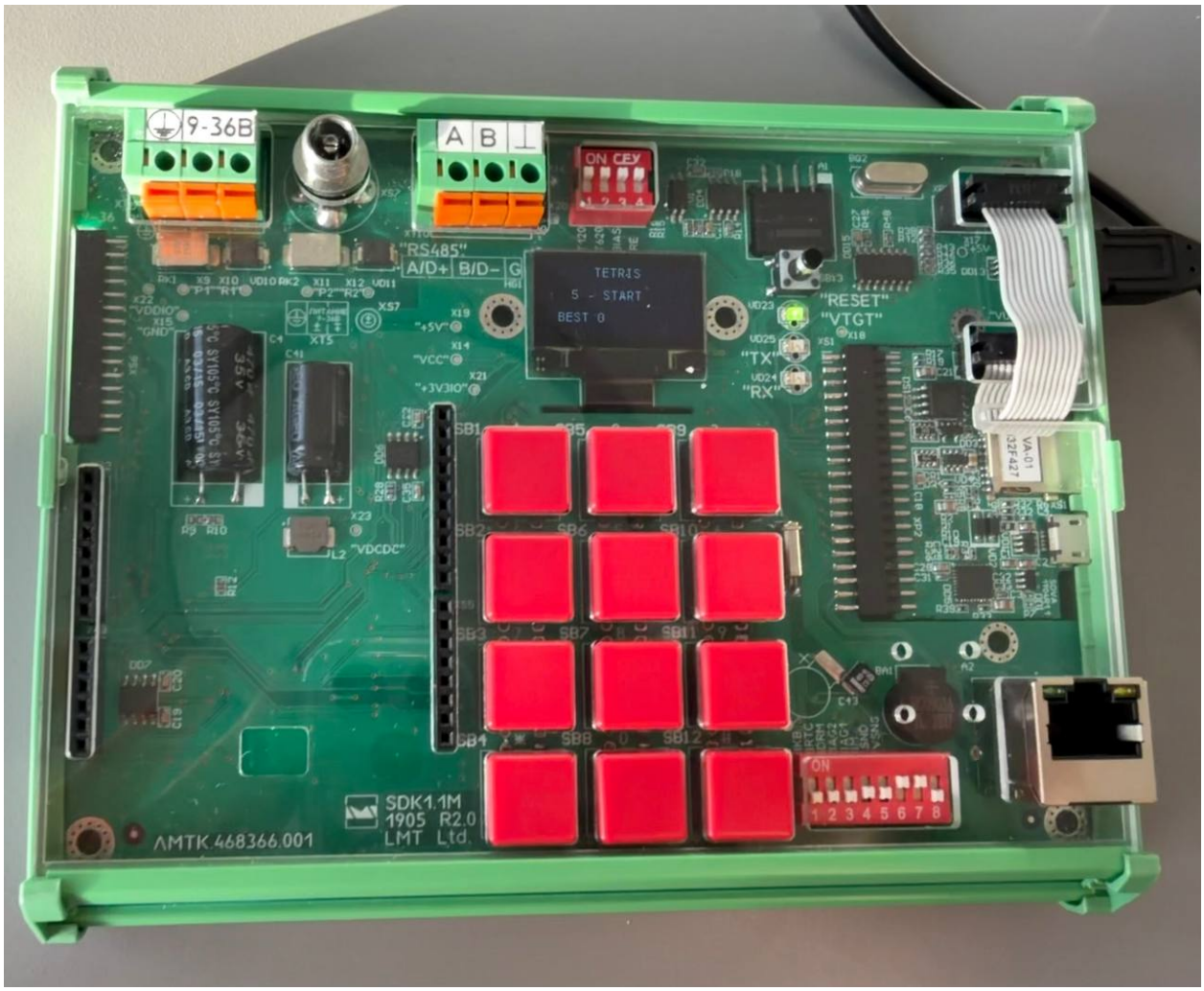


Рисунок 12 — Возврат в меню после завершения

Код

Ссылка на репозиторий:
https://github.com/yaqwi/itmo/tree/main/3%20курс/встроенные%20системы/мастер-класс%203/STM32CubeIDE